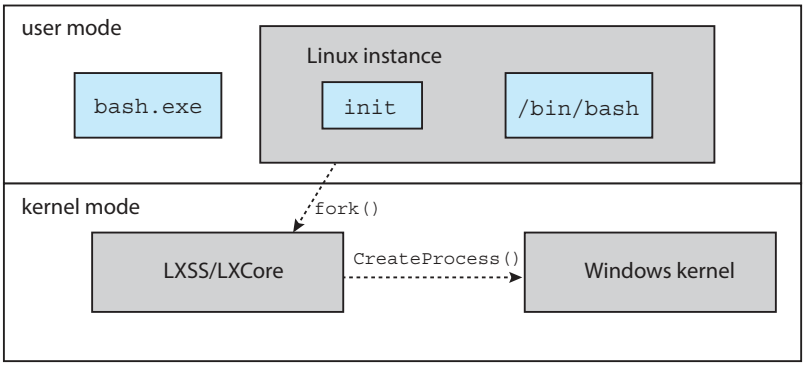
ویندوز پیش می‌فرستد. در برخی شرایط، لینوکس و ویندوز دارای فراخوانی‌های سیستمی هستند که مشابه هستند اما یکسان نیستند. موقعی که این حالت پیش می‌آید، LXSS گوشه‌ای از کارکرد را فراهم ساخته و فراخوانی سیستمی مشابه ویندوز را احضار می‌کند تا بقیه‌ی کارکرد را فراهم کند. فراخوانی سیستمی ()fork لینوکس، نمودی از این حالت است: فراخوانی سیستمی ()CreateProcess ویندوز مشابه ()fork است اما دقیقاً همان کارکرد را ندارد. زمانی که ()fork در WSL احضار می‌شود، سرویس LXSS بخشی از کار آغازین ()fork را انجام داده و سپس ()CreateProcess را فرا می‌خواند تا باقی کار را انجام بدهد. شکل زیر، رفتار پایه‌ای WSL را نمایش می‌دهد.



۲-۹ ساخت و راه‌اندازی یک سیستم‌عامل

طراحی، کدنویسی، و پیاده‌سازی یک سیستم‌عامل مخصوصاً برای یک پیکربندی ماشین خاص امکان‌پذیر است. با این وجود، سیستم‌عامل‌ها عمدتاً برای اجرا بر روی هر کلاسی از ماشین‌ها با پیکربندی‌های جانبی گوناگونی طراحی می‌شوند.

۲-۹-۱ تولید سیستم‌عامل

در بیشتر موارد، موقعی که یک سیستم کامپیوتری خریداری می‌شود یک سیستم‌عامل پیشاپیش نصب شده دارد. برای مثال، شما ممکن است یک لپ‌تاپ جدیدی با ویندوز یا macOS از پیش نصب شده خریده باشید. اما فرض کنید که شما می‌خواهید سیستم‌عامل از پیش نصب شده را جایگزین کرده یا سیستم‌عامل‌های دیگری اضافه کنید. یا فرض کنید که یک کامپیوتری بدون یک سیستم‌عامل می‌خرید. در این شرایط، شما گزینه‌های معدودی دارید تا به‌عنوان سیستم‌عامل مناسب بر روی کامپیوتر نصب کرده و آن برای استفاده پیکربندی کنید.

اگر شما یک سیستم‌عامل را به‌کل تولید کنید (یا بسازید)، بایستی گام‌های زیر را دنبال کنید:

۱-کد اصلی سیستم‌عامل را بنویسید (یا کد اصلی از پیش نوشته شده را بدست آورید).

۲-سیستم‌عامل را برای سیستمی که روی آن اجرا خواهد شد پیکربندی کنید.

۳-سیستم‌عامل را کامپایل کنید.

۴-سیستم‌عامل را نصب کنید.

۵-کامپیوتر و سیستم‌عامل جدید آن را بوت کنید.

پیکربندی سیستم، مشمول توصیف امکاناتی است که لحاظ خواهند شد، و این موضوع بسته به سیستم‌عامل فرق می‌کند. نوعاً، پارامترهایی که چگونگی پیکربندی سیستم را بیان می‌کنند در یک فایل پیکربندی از یک نوعی ذخیره می‌شود و همین که این فایل ایجاد شد، می‌تواند به‌شیوه‌های متعددی استفاده شود.

از یک سو، مدیر سیستم می‌تواند با استفاده از آن کپی کد اصلی سیستم‌عامل را تغییر بدهد. سپس، سیستم‌عامل به صورت کامل کامپایل می‌شود (معروف به **ساخت سیستم**). اعلان داده‌ها، مقداردهی‌های اولیه و ثابت‌ها در کنار شرایط کامپایل، خروجی نسخه‌ی مقصد سیستم‌عامل را تولید می‌کند که مطلوب توصیف سیستم فایل پیکربندی می‌باشد.

توصیف سیستم در سطحی نه چندان مطلوب می‌تواند به انتخاب ماژول‌های از پیش کامپایل شده از یک کتابخانه‌ی موجود منجر شود. این ماژول‌ها به همدیگر متصل می‌شوند تا سیستم‌عامل تولیدی را شکل بدهند. این فرایند به کتابخانه این امکان را می‌دهد که درایور دستگاه‌ها را برای تمامی دستگاه‌های I/O که پشتیبانی می‌شوند لحاظ کند، اما تنها موارد لازم انتخاب شده و به سیستم‌عامل متصل می‌شوند. چون سیستم دوباره کامپایل نمی‌شود، تولید سیستم سریع‌تر انجام می‌شود، اما سیستم حاصل ممکن است بیشتر از معمول آن کلی باشد و ممکن است پیکربندی‌های سخت‌افزاری متفاوت را پشتیبانی نکند.

از سویی دیگر، می‌توان سیستمی برپا کرد که کاملاً ماژولار است. در اینجا، انتخاب به‌جای کامپایل یا زمان اتصال در زمان اجرا صورت می‌گیرد. تولید سیستم، به‌سادگی، انتخاب پارامترهایی است که پیکربندی سیستم را تشریح می‌کنند.

تفاوت‌های اصلی این رویکردها در اندازه و عمومیت سیستم تولید شده و راحتی اصلاح آن با تغییرات پیکربندی سخت‌افزار است. در سیستم‌های تعبیه‌شده مرسوم است که رویکرد اول را برداشته و یک سیستم‌عاملی برای یک پیکربندی سخت‌افزاری خاص ایستا ایجاد شود. با این وجود، اکثر سیستم‌عامل‌های مدرن که از کامپیوتری رومیزی و لپ‌تاپ پشتیبانی می‌کنند در کنار دستگاه‌های موبایل، رویکرد دوم را تطبیق داده‌اند. یعنی، سیستم‌عامل همچنان برای یک پیکربندی سخت‌افزاری خاصی تولید می‌شود، اما استفاده از تکنیک‌هایی همچون ماژول‌های بارشدنی پویا، پشتیبانی ماژولاری برای تغییرات پویا به سیستم فراهم می‌کند.

اکنون ساخت یک سیستم لینوکس را به‌کل بیان می‌کنیم که نوعاً انجام گام‌های زیر ضرورت دارد:

۱-کد اصلی لینوکس را از http://www.kernel.org دانلود کنید.

۲-هسته را با استفاده از فرمان «make menuconfig» پیکربندی کنید. این مرحله فایل پیکربندی config. را تولید می‌کند.

۳-هسته‌ی اصلی را با استفاده از فرمان «make» کامپایل کنید. فرمان make هسته را بر اساس پارامترهای پیکربندی مشخص شده در فایل config. کامپایل کرده، فایل vmlinuz را تولید می‌کند که تصویر هسته است.

۴-ماژول‌های هسته را با استفاده از فرمان «make modules» کامپایل کنید. درست مثل

کامپایل هسته، کامپایل ماژول به پارامترهای پیکربندی توصیف شده در فایل config. بستگی دارد.

۵-از فرمان «make modules_install» استفاده کنید تا ماژول‌های هسته را در vmlinuz نصب کنید.

۶-هسته‌ی جدید را با واردکردن فرمان «make install» بر روی سیستم نصب کنید.

موقعی که سیستم دومرتبه راه‌اندازی می‌شود، اجرای این سیستم‌عامل جدید را آغاز خواهد کرد.

در روشی دیگر، این امکان وجود دارد که یک سیستم موجود را با نصب یک ماشین مجازی لینوکس اصلاح کرد. این موضوع به سیستم‌عامل میزبان (همچون ویندوز یا macOS) اجازه می‌دهد تا لینوکس را اجرا کند. (ما مجازی‌سازی را در بخش ۱-۷ معرفی کردیم و موضوع را با جزئیات بیشتر در فصل ۱۸ پوشش می‌دهیم.)

گزینه‌های معدودی برای نصب لینوکس به‌صورت یک ماشین مجازی وجود دارد. یک سناریو این است که یک ماشین مجازی را به‌کل ساخت. این گزینه شبیه به ساخت یک سیستم لینوکس کامل است؛ با این وجود، سیستم‌عامل نیازی به کامپایل شدن ندارد. رویکرد دیگر استفاده از ابزار ماشین مجازی لینوکس است، سیستم‌عاملی که از قبل ساخته و پیکربندی شده است. این گزینه به‌سادگی لازم دارد که ابزار دانلود شده و با استفاده از نرم‌افزار مجازی‌سازی، همچون Virtualbox یا VMware نصب شود. برای مثال، نویسندگان همین کتاب برای ساخت سیستم‌عاملی که در قالب ماشین مجازی تدارک دیده شده کارهای زیر را انجام دادند:

۱- دانلود تصویر Ubunto ISO از سایت https://www.ubuntu.com/

۲- هدایت و راهنمایی نرم‌افزار ماشین مجازی VirtualBox برای استفاده از ISO به‌عنوان یک رسانه‌ی قابل بوت و راه‌اندازی ماشین مجازی

۳- پاسخ‌گویی به پرسش‌های فرایند نصب و سپس نصب و بوت سیستم‌عامل به‌عنوان یک ماشین مجازی

۲-۹-۲ راه‌اندازی سیستم

بعد از این که سیستم‌عامل تولید می‌شود، بایستی جهت استفاده در یک سخت‌افزار قرار بگیرد. اما، سخت‌افزار از کجا بداند که هسته در کجاست یا چگونه هسته را بار کند؟ فرایند شروع به کار کامپیوتر با بارگذاری هسته به *راه‌اندازی*[1] سیستم معروف است. در اکثر سیستم‌های کامپیوتری، فرایند راه‌اندازی به‌صورت زیر دنبال می‌شود:

۱-تکه‌کد کوچکی تحت عنوان **برنامه‌ی بالاآورنده**[2] یا **بارگذار بوت**[3] هسته را پیدا می‌کند.

۲-هسته به داخل حافظه بارگذاری شده و شروع به اجرا می‌کند.

۳-هسته سخت‌افزار را مقداردهی اولیه می‌دهد.

1. Booting
2. Bootstrap program
3. Boot loader

۴-سیستم‌فایل ریشه سوار می‌شود.

ما در این بخش، فرایند بوت را با جزئیات بیشتر تشریح می‌کنیم.

برخی سیستم‌های کامپیوتری از یک فرایند بوت چندمرحله‌ای استفاده می‌کنند: موقعی که کامپیوتر برای اولین بار روشن می‌شود، یک بارگذار بوت کوچکی که در سفت‌افزار نامیرایی تحت عنوان **BIOS** قرار دارد، اجرا می‌شود. این بارگذار بوت آغازین معمولاً کار خاصی به‌غیر از بارگذاری یک بارگذار بوت دوم انجام نمی‌دهد که در یک مکان ثابتی از دیسک به‌نام **بلاک بوت**[1] قرار دارد. برنامه‌ای که در بلاک بوت ذخیره شده است می‌تواند به‌حد کافی پیچیده باشد تا کل سیستم‌عامل را به داخل حافظه بارگذاری کرده و اجرایش را شروع کند. در بیشتر موارد، کد ساده‌ای است (تا این که در یک بلاک دیسک تکی جا بگیرد) و تنها آدرس دیسک و طول باقیمانده‌ی برنامه‌ی بالاآورنده را می‌داند.

بسیاری از سیستم‌های کامپیوتری اخیر فرایند بوت مبتنی بر BIOS را با **UEFI** (رابط سفت‌افزار گسترش‌پذیر یکه[2]) جایگزین کرده‌اند. UEFI مزایای متعددی نسبت به BIOS دارد، اعم از پشتیبانی بهتر سیستم‌های ۶۴ بیتی و دیسک‌های بزرگ‌تر. شاید بزرگ‌ترین مزیت این باشد که UEFI یک مدیر بوت منفرد کامل است و لذا سریع‌تر از فرایند بوت چندمرحله‌ای BIOS می‌باشد.

فارغ از این که راه‌اندازی از BIOS یا UEFI باشد، برنامه‌ی بالاآورنده می‌تواند کارهای گوناگونی را انجام دهد. علاوه بر بارگذاری فایل شامل برنامه‌ی هسته به داخل حافظه، برنامه‌های تشخیص[3] را هم اجرا می‌کند تا وضعیت ماشین را تعیین کند—برای مثال، بازرسی حافظه و CPU و کشف دستگاه‌های متصل. اگر مرحله‌ی تشخیص پشت سر گذاشته شود، برنامه می‌تواند مراحل راه‌اندازی را ادامه دهد. برنامه‌ی بالاآورنده می‌تواند تمامی وجوه سیستم را از ثبات‌های پردازنده گرفته تا کنترلر دستگاه‌ها و محتویات حافظه‌ی اصلی، مقدار اولیه دهد. همان لحظه یا کمی بعد، سیستم‌عامل را شروع کرده و سیستم‌فایل ریشه را سوار می‌کند. تنها در این لحظه است که گفته می‌شود سیستم **در حال اجرا** می‌باشد.

GRUB یک برنامه‌ی بالاآورنده‌ی متن‌باز برای سیستم‌های لینوکس و یونیکس است. پارمترهای بوت سیستم در یک فایل پیکربندی GRUB مشخص می‌شوند که در بالاآمدن سیستم بارگذاری می‌شود. GRUB انعطاف‌پذیر است و امکان ایجاد تغییرات را در زمان بوت فراهم می‌کند، اعم از تغییر پارامترهای هسته و حتی انتخاب از میان هسته‌های متفاوتی که می‌توانند بوت شوند. به‌عنوان یک مثال، موارد زیر، پارامترهای هسته از یک فایل خاص /proc/cmdline لینوکس هستند که در زمان بوت استفاده می‌شود:

```
BOOT_IMAGE=/boot/vmlinuz-4.4.0-59-generic
root=UUID=5f2e2232-4e47-4fe8-ae94-45ea749a5c92
```

1. Boot block
2. Unified Extensible Firmware Interface (UEFI)
3. Diagnostics

BOOT_IMAGE نام تصویر هسته‌ای که قرار است به‌داخل حافظه بارگذاری شود، و root، شناسه‌ی یکتای سیستم‌فایل ریشه را مشخص می‌کند.

به‌منظور صرفه‌جویی در فضا در کنار کاهش زمان بوت، تصویر هسته‌ی لینوکس، یک فایل فشرده است که بعد از این که به‌داخل حافظه بارگذاری شد از حالت فشرده خارج می‌شود. در طی فرایند بوت، بارگذار بوت نوعاً یک سیستم‌فایل RAM موقتی، تحت عنوان initramfs ایجاد می‌کند. این سیستم‌فایل شامل درایورها و ماژول‌های هسته‌ی ضروری است که بایستی نصب شوند تا از سیستم‌فایل ریشه‌ی حقیقی پشتیبانی کنند (که در حافظه‌ی اصلی نیست). همین که هسته شروع کرد و درایورهای لازم نصب شدند، هسته سیستم‌فایل ریشه را از محل موقتی RAM به محل مناسب سیستم‌فایل ریشه بر می‌گرداند. در نهایت، لینوکس پردازه‌ی systemd، پردازه‌ی آغازین سیستم را ایجاد می‌کند، و سپس سرویس‌های دیگر را شروع می‌کند (برای مثال، کارگزار وب و/یا پایگاه‌داده). سرانجام، سیستم برای کاربر اعلان login را نمایش می‌دهد. در بخش ۱۱-۵-۲، فرایند بوت ویندوز را بیان می‌کنیم.

می‌ارزد که توجه داشته باشید راهکار بوت مستقل از بارگذار بوت نیست. بنابراین، نسخه‌های خاصی از بارگذار بوت GRUB و UEFI وجود دارند و سفت‌افزار بایستی بداند که کدام بارگذار بوت خاص قرار است استفاده شود.

فرایند بوت سیستم‌های موبایل اندکی با کامپیوترهای شخصی متعارف فرق دارد. برای مثال، هرچند که اندروید هسته‌اش مبتنی بر لینوکس است از GRUB استفاده نمی‌کند و در عوض، این کار را برای فروشندگان باقی می‌گذارد تا بارگذارهای بوت خودشان را فراهم کنند. رایج‌ترین بارگذار بوت اندروید LK (اختصار «هسته‌ی کوچک[1]») است. سیستم‌های اندروید، تصویر هسته‌ی فشرده‌ی یکسانی مثل لینوکس، همین‌طور یک سیستم‌فایل RAM آغازین استفاده می‌کنند. با این وجود، در حالی که لینوکس، به‌محض بارگذاری تمامی درایورهای ضروری، initramfs را کنار می‌گذارد، اندروید initramfs را به‌عنوان سیستم‌فایل ریشه‌ی دستگاه حفظ می‌کند. همین که هسته بارگذاری شده و سیستم‌فایل ریشه سوار شد، اندروید پردازه‌ی init را شروع کرده و قبل از نمایش صفحه‌نمایش خانگی، تعدادی سرویس ایجاد می‌کند.

در نهایت، بارگذارهای بوت اکثر سیستم‌عامل‌ها—اعم از ویندوز، لینوکس، و macOS، در کنار هر دوی iOS و اندروید—بوت را به‌شکل **مد ترمیم**[2] یا **مد تک‌کاربر** فراهم می‌کنند تا موارد سخت‌افزاری را تشخیص بدهند، سیستم‌فایل‌های خراب را تعمیر کنند، و حتی سیستم‌عامل را دومرتبه نصب کنند. علاوه بر خرابی‌های سخت‌افزاری، سیستم‌های کامپیوتری می‌توانند از خطاهای نرم‌افزاری و کارایی ضعیف سیستم‌عامل رنج ببرند که در بخش بعدی مد نظر قرار می‌دهیم.

۲-۱۰ عیب‌یابی سیستم‌عامل

عیب‌یابی را هر از چندی در این فصل متذکر شدیم. در اینجا، از نزدیک به این موضوع نگاه

1. Little kernel (LK)
2. Recovery mode

می‌کنیم. به صورت وسیع‌تر، **عیب‌یابی** عمل یافتن و ترمیم خطاها یا **عیب‌ها** در یک سیستم، در هر دوی سخت‌افزار و نرم‌افزار است. عیب‌یابی هم به دنبال یافتن خطاهای سخت‌افزاری و نرم‌افزاری و نیز ترمیم آنهاست. مسائل کارایی عیب محسوب می‌شوند، لذا عیب‌یابی می‌تواند **تنظیم کارایی**[1] را هم در بر بگیرد که در صدد است تا با برداشتن **گلوگاه‌های** پردازشی، کارایی را بهبود ببخشد. در این بخش، فرایند عیب‌یابی و خطاهای هسته و مسائل کارایی را بررسی می‌کنیم. عیب‌یابی سخت‌افزاری خارج از حوزه‌ی مباحث این کتاب است.

۲-۱۰-۱ تحلیل خرابی

اکثر سیستم‌عامل‌ها با از کار افتادن یک پردازه، ضمن نوشتن اطلاعات خطا در یک **فایل سابقه** به مدیران یا کاربران سیستم هشدار می‌دهند که مسأله‌ای رخ داده است. سیستم‌عامل می‌تواند یک **رونوشت حافظه**[2]–تصویری از حافظه‌ی پردازه–نیز بگیرد و در فایلی برای تحلیل آتی ذخیره کند. (در روزهای آغازین محاسبات، به حافظه «core» اطلاق می‌شد.) برنامه‌های در حال اجرا و رونوشت‌های حافظه‌ی آنها می‌تواند توسط یک **عیب‌یاب**[3] پرس‌وجو شود که این کار، امکان بررسی کد و حافظه‌ی یک پردازه را در زمان خرابی به یک برنامه‌نویس می‌دهد.

عیب‌یابی کد پردازه‌ی سطح کاربر به عنوان یک چالش مطرح است. عیب‌یابی هسته‌ی سیستم‌عامل، به خاطر اندازه و پیچیدگی هسته، کنترل سخت‌افزار توسط آن و نبود ابزار عیب‌یابی آن در سطح کاربر حتی پیچیده‌تر هم می‌شود. یک خرابی در هسته، یک **شکست**[4] نامیده می‌شود. موقعی که یک شکست رخ می‌دهد، اطلاعات خطا در یک فایل سابقه نوشته می‌شود و وضعیت حافظه در یک **رونوشت شکست**[5] ذخیره می‌گردد.

عیب‌یابی سیستم‌عامل و عیب‌یابی پردازه، در بیشتر موارد، به واسطه‌ی طبیعت بسیار متفاوت این دو کار، از ابزار و تکنیک‌های متفاوتی استفاده می‌کنند. در نظر بگیرید که بعد از وقوع یک خرابی هسته در کد سیستم‌فایل برای هسته مخاطره‌آمیز است که بخواهد حالت خود را پیش از راه‌اندازی مجدد در فایلی از سیستم‌فایل ذخیره نماید. یک تکنیک مرسوم آن است که حالت حافظه‌ی هسته در بخشی از دیسک ذخیره شود که برای این منظور کنار گذاشته شده و فاقد سیستم‌فایل است. اگر هسته یک خطای غیرقابل ترمیم تشخیص بدهد، کل محتویات حافظه یا حداقل بخش‌هایی از حافظه‌ی سیستم را که مختص هسته است در سطح دیسک می‌نویسد. زمانی که سیستم دومرتبه راه‌اندازی می‌شود، پردازه‌ای اجرا می‌شود تا داده‌ها را از آن ناحیه‌ی دیسک جمع‌آوری کرده و جهت تحلیل بیشتر در میان سیستم‌فایل در یک فایل رونوشت شکست بنویسد. روشن است که این قبیل راهبردها برای عیب‌یابی پردازه‌های معمول سطح کاربر ضرورتی نخواهند داشت.

1. Performance tuning
2. Core dump
3. Debugger
4. Crash
5. Crach dump

قانون Kernighan

«عیب‌یابی در درجه‌ی اول، دو مرتبه از نوشتن کد دشوارتر است. بنابراین، کد را با هر زکاوت ممکن هم که نوشته باشید، طبق تعریف، در عیب‌یابی آن به اندازه‌ی کافی زرنگ نیستید.»

۲-۱۰-۲ کنترل و تنظیم کارایی

پیشتر متذکر شدیم که تنظیم کارایی در صدد است با برداشتن گلوگاه‌های پردازشی، کارایی را بهبود ببخشد. به منظور شناسایی گلوگاه‌ها، بایستی بتوانیم بر کارایی سیستم نظارت کنیم. بنابراین، سیستم‌عامل بایستی یک سری ابزار برای محاسبه و نمایش رفتار سیستم داشته باشد. ابزار ممکن است بر اساس این که مشاهدات را *به‌ازای هر پردازه* یا *در سطح سیستم* فراهم می‌کنند مشخص شوند. ابزار برای ایجاد این مشاهدات ممکن است یکی از دو رویکرد را استفاده کنند—*شمارنده‌ها* یا *ردیابی*[1].

۲-۱۰-۲-۱ شمارنده‌ها

سیستم‌عامل‌ها رد فعالیت سیستم را از طریق تعدادی شمارنده نگه می‌دارند، همچون تعداد فراخوانی‌های سیستمی انجام شده یا تعداد عملیات انجام شده به یک دستگاه شبکه یا دیسک. موارد زیر مثال‌هایی از ابزار لینوکس هستند که از شمارنده‌ها استفاده می‌کنند.

به‌ازای هر پردازه

- ps—اطلاعاتی برای یک پردازه یا گزینشی از پردازه‌ها گزارش می‌دهد
- top—آمار بلادرنگی برای پردازه‌های فعلی گزارش می‌دهد

در سطح سیستم

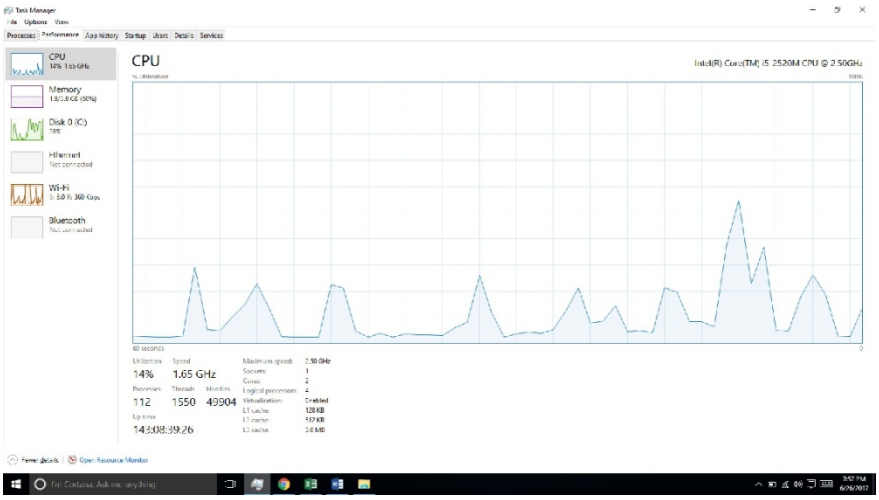
- vmstat—آمار استفاده از حافظه را گزارش می‌دهد
- netstat—آمار رابط‌های شبکه را گزارش می‌دهد
- iostat—استفاده‌ی I/O به دیسک‌ها را گزارش می‌دهد

اکثر ابزار مبتنی بر شمارنده در سیستم‌های لینوکس، آمار را از سیستم‌فایل proc/ می‌خوانند. proc/ یک «شبه» سیستم‌فایل است که تنها در حافظه‌ی هسته وجود دارد و اصولاً برای پرس‌وجوی آمار پردازه‌ای، همین‌طور هسته استفاده می‌شود. سیستم‌فایل proc/ به‌صورت یک سلسله‌مراتب دایرکتوری با پردازه (یک مقدار عددی صحیح که به هر پردازه منتسب می‌شود) که به‌صورت یک زیردایرکتوری در زیر proc/ ظاهر می‌شود سازمان‌دهی می‌شود. برای مثال، وارده‌ی دایرکتوری proc/2155/ آمار مربوط به پردازه را برای پردازه‌ای با یک شناسه‌ی ۲۱۵۵ شامل خواهد بود. وارده‌های proc/ برای آمار گوناگون هسته نیز وجود دارد. در هر دوی این

1. Tracing

فصل و فصل ۳، ما پروژه‌های برنامه‌نویسی تدارک دیده‌ایم که شما سیستم‌فایل proc/ را ایجاد و دسترسی خواهید کرد.

سیستم‌های ویندوز **مدیر وظیفه‌ی ویندوز** را فراهم می‌کنند، ابزاری که شامل اطلاعاتی برای برنامه‌های فعلی، همین‌طور پردازه‌ها، استفاده از CPU و حافظه، و آمار شبکه می‌باشد. یک تصویر لحظه‌ای از مدیر وظیفه در ویندوز ۱۰ در شکل ۲–۱۹ نشان داده شده است.



شکل ۲–۱۹ مدیر وظیفه‌ی ویندوز ۱۰

۲–۱۰–۳ ردیابی

در حالی که ابزار مبتنی بر شمارنده به‌سادگی راجع به مقدار فعلی آمار خاصی پرس‌وجو می‌کنند که توسط هسته حفظ می‌شود، ابزار ردیابی داده‌هایی را برای یک رویداد خاص جمع‌آوری می‌کنند—همچون گام‌های مشمول در یک احضار فراخوانی سیستمی.

موارد زیر، مثال‌هایی از ابزار لینوکس هستند که رویدادها را ردیابی می‌کنند:

به‌ازای هر پردازه

- strace—فراخوانی‌های سیستمی را ردیابی می‌کند که توسط یک پردازه احضار شده‌اند
- gdb—یک عیب‌یاب سطح کد اصلی

در سطح سیستم

- perf—مجموعه‌ای از ابزار کارایی لینوکس
- tcpdump—بسته‌های شبکه را جمع‌آوری می‌کند

آسان‌تر ساختن درک، عیب‌یابی، و تنظیم سیستم‌عامل‌ها در اجرای آن‌ها یکی از حوزه‌های فعال تحقیق و تجربه می‌باشد. نسل جدیدی از ابزار تحلیل کارایی با کمک هسته، بهبودهای قابل

توجهی در چگونگی تحقق این هدف در پی داشته است. در ادامه، ما BCC را بحث می‌کنیم که مجموعه ابزاری برای ردیابی پویای هسته در لینوکس می‌باشد.

BCC ۴-۱۰-۲

عیب‌یابی تعاملات میان کد سطح کاربر و هسته بدون مجموعه ابزاری که هر دو مجموعه کد را درک کرده و تعاملات آن‌ها را وساطت می‌کند تقریباً ناممکن است. برای این که مجموعه ابزار ذکر شده حقیقتاً مفید باشند بایستی قادر به عیب‌یابی هر ناحیه از یک سیستم باشند، اعم از نواحی که با ذهنیت عیب‌یابی نوشته نشده‌اند، و این کار را بدون تأثیرگذاری بر قابلیت اطمینان سیستم انجام بدهند. این مجموعه ابزار بایستی حداقل پیامد منفی بر کارایی را داشته باشند— در حد ایده‌آل آن بایستی موقعی که استفاده ندارند هیچ پیامدی نداشته و در طی استفاده هم پیامد مناسبی داشته باشند. مجموعه ابزار BCC این نیازمندی‌ها را برآورده ساخته و یک محیط پویا، امن، از پیامد منفی پایین عیب‌یابی برخوردارند.

BCC[1] (مجموعه کامپایلر BPF) یک مجموعه ابزار غنی است که امکانات ردیابی را برای سیستم‌های لینوکس فراهم می‌کند. BCC یک رابط خط مقدم[2] برای ابزار eBPF[3] (فیلتر بسته‌ی توسعه‌یافته‌ی برکلی) می‌باشد. فناوری BPF در اوایل دهه‌ی ۱۹۹۰ برای فیلترکردن ترافیک در یک شبکه‌ی کامپیوتری توسعه داده شد. BPF «توسعه‌یافته» (eBPF) امکانات گوناگونی را به BPF افزود. برنامه‌های eBPF به زیرمجموعه‌ای از C نوشته شده و به دستورالعمل‌های eBPF کامپایل می‌شوند که می‌توانند به‌صورت پویا در سیستم در حال اجرای لینوکس درج شوند. دستورالعمل‌های eBPF می‌توانند برای گرفتن رویدادهای خاصی (همچون یک فراخوانی سیستمی خاصی که احضار می‌شود) یا کنترل کارایی سیستم (همچون زمان لازم برای انجام I/O دیسک) استفاده شوند. برای اطمینان از این که دستورالعمل‌های eBPF خوش‌رفتار هستند، یک **وارسی**[4] را پشت سر می‌گذارند قبل از این که در هسته‌ی در حال اجرای لینوکس درج شوند. وارس بررسی می‌کند تا اطمینان حاصل کند که دستورالعمل‌ها کارایی یا امنیت سیستم را تحت تأثیر قرار نمی‌دهند.

هرچند که eBPF یک مجموعه‌ی غنی از امکاناتی را برای ردیابی در میان هسته‌ی لینوکس فراهم می‌کند، سابقاً نوشتن برنامه‌ها با استفاده از رابط C آن بسیار دشوار بود. BCC توسعه داده شد تا نوشتن ابزار با استفاده از eBPF با فراهم‌ساختن یک رابط خط مقدم در پایتون ساده‌تر شود. یک ابزار BCC در پایتون نوشته شده و کد C کار می‌گذارد که با با اسباب eBPF ارتباط برقرار می‌کند که به‌نوبت خود با هسته ارتباط برقرار می‌سازند. ابزار BCC برنامه‌ی C را نیز به دستورالعمل‌های eBPF کامپایل کرده و با استفاده از پروب‌ها[5] و یا نقاط ردیابی[6] در هسته درج می‌کند، دو تکنیکی که رویدادهای ردیابی را در هسته‌ی لینوکس امکان‌پذیر می‌سازند.

1. BPF Compiler Collection (BCC)
2. Front-end
3. extended Berkeley Packet Filter (eBPF)
4. Verifie
5. Probes
6. Tracepoints

توضیح جزئیات نوشتن ابزار سفارشی BCC فراتر از حوزه و قلمرو این کتاب است، اما بسته‌ی BCC (که در روی ماشین مجازی لینوکسی که ما تدارک دیده‌ایم نصب شده است) تعدادی از ابزار موجودی را فراهم می‌کند که نواحی متعددی از فعالیت یک هسته‌ی در حال اجرای لینوکس را کنترل می‌کنند. به‌عنوان یک مثال، ابزار BCC disksnoop فعالیت I/O دیسک را ردیابی می‌کند. واردکردن فرمان

```
./disksnoop.py
```

خروجی نمونه‌ی زیر را تولید می‌کند:

```
TIME(s)             T          BYTES          LAT(ms)
1946.29186700       R          8              0.27
1946.33965000       R          8              0.26
1948.34585000       W          8192           0.96
1950.43251000       R          4096           0.56
1951.74121000       R          4096           0.35
```

این خروجی، زمانی را که عملیات I/O تکمیل شده است، عملیات I/O خواندن یا نوشتن بوده، و چه تعداد بایت در I/O منتقل شده‌اند در اختیار ما می‌گذارد. ستون آخری فاصله‌ی زمانی I/O را (که با دیرکرد یا LAT بیان شده) با میلی‌ثانیه نشان می‌دهد.

بسیاری از ابزاری که توسط BCC فراهم شده‌اند می‌توانند برای کاربردهای خاصی استفاده شوند، همچون پایگاه‌داده‌های MySQL، همین‌طور برنامه‌های جاوا و پایتون. می‌توان برای کنترل فالیت یک پردازه‌ی خاصی پروب هم گذاشت. برای مثال، فرمان

```
./opensnoop -p 1225
```

فراخوانی‌های سیستمی ()open را که صرفاً توسط پردازه‌ای با شناسه‌ی ۱۲۲۵ انجام شده‌اند، ردیابی می‌کند.

چیزی که BCC را مخصوصاً قدرتمند می‌سازد این است که ابزار آن می‌توانند در سیستم‌های تولیدی سرپا استفاده شوند که در حال اجرای برنامه‌های بحرانی هستند، بدون این که برای سیستم ضرری داشته باشند. این موضوع مخصوصاً برای مدیران سیستم مفید است که بایستی کارایی سیستم را کنترل کنند تا گلوگاه‌های ممکن یا تخطی‌های امنیتی را شناسایی کنند. شکل ۲-۲۰ بازه‌ی وسیعی از ابزارهایی که در حال حاضر توسط BCC و eBPF فراهم شده و توانایی آن‌ها در ردیابی هر ناحیه‌ای از سیستم‌عامل لینوکس را نشان می‌دهد. BCC یک فناوری است که به‌سرعت با امکانات جدیدی که به‌صورت مستمر در حال افزوده شدن است در حال تغییر می‌باشد.